

450. Delete Node in a BST

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 29, 2026

Problem Summary

Given the root of a BST and a key, delete the node with that key and return the updated root.

Approach

Search for the node to delete. Once found, handle three cases: leaf (return null), one child (return that child), two children (replace with in-order successor — leftmost node in right subtree — then delete successor from right subtree).

Algo

1. If `root == null` return null.
2. If `key < root.val` → `root.left = deleteNode(root.left, key)`.
3. If `key > root.val` → `root.right = deleteNode(root.right, key)`.
4. Else (found):
 - `root.left == null` → return `root.right`.
 - `root.right == null` → return `root.left`.
 - Both exist → find successor = `findMin(root.right)`, set `root.val = successor.val`, delete successor from right subtree.
5. Return root.

Observations

- `findMin` iterates left until `root.left == null` — always iterative, never recursive.
- Could also use in-order **predecessor** (rightmost in left subtree) — both are valid.
- The recursive `deleteNode(root.right, successor.val)` is guaranteed to hit the one-child or leaf case since successor has no left child by definition.

Key Insights

- Three deletion cases: leaf, one child, two children — handle in increasing complexity order.
- Cases 1 and 2 are handled by the same two lines — returning the non-null child (or null for leaf) covers both.
- In-order successor (leftmost in right subtree) maintains BST property — it's larger than entire left subtree and smaller than rest of right subtree.
- Don't actually remove and relink the successor node — just copy its value and delete it recursively.

Time Complexity & Space Complexity

$O(h)$ — search path + `findMin` path, both $O(h)$.

$O(h)$ — recursion stack.

Problem List

450. Delete Node in a BST

Solved

Medium

Topics

Companies

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the *root node reference* (possibly updated) of the BST.

Basically, the deletion can be divided into two stages:

- Search for a node to remove.
- If the node is found, delete the node.

Example 1:

Input: root = [5,3,6,2,4,null,7], key = 3
Output: [5,4,6,2,null,null,7]
Explanation: Given key to delete is 3. So we find the node with value 3 and delete it. One valid answer is [5,4,6,2,null,null,7], shown in the above BST. Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.

Accepted

All Submissions

Accepted 95 / 95 testcases passed

Sahil Khan submitted at Jul 29, 2026 23:51

7583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials —...

Runtime

0 ms Beats 100.00%

Memory

47.06 MB Beats 97.80%

Code | Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

root = [5,3,6,2,4,null,7]